Jaypee Institute of Information Technology, Noida

DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING AND INFORMATION TECHNOLOGY

Project Title: GitLite

Submitted By:

Sriza Goel(2401030094)

Toyesh Gupta(2401030081)

Khushii Duggal(2401030085)

Submitted To:

Dr. Alka Singhal

Dr. Taj Alam

Mr. Amitesh

Course Name: Competitive Programming Lab

Course Code: 24B15CS224
Program: B. Tech. CSE 2
nd Year 4 th Semester

2025 – 2026

TABLE OF CONTENTS

Topic	Page Number
1. Summary 1.1 Brief overview of the project 1.2 Objectives	1
2. Introduction 2.1 Background and Context 2.2 Problem Statement 2.3 Project Objectives 2.4 Scope of the Project	2-4
3. System Requirements 3.1 Functional Requirements	4-5
4. Design and Implementation 4.1 Algorithms Implementation 4.2 Algorithms Complexity 4.3 UML Diagram 4.4 Output	10-13
5. Conclusion 5.1 Summary & achievement of Objectives 5.2 Future Work and Recommendations	7-11
6. References	12

1. Summary

1.1 Brief Overview of the project

GitLite is a custom-designed system developed to manage, track, analyze, and visualize changes in files over time. It integrates a high-performance backend written in C++ with a modern web-based interface that allows users to interact with and understand the system in a highly intuitive manner.

The system goes beyond basic version tracking by incorporating advanced features such as:

- Line-based difference analysis
- Delta compression for storage optimization
- Content-based addressing using hashing
- Visualization of internal storage and reconstruction processes
- Graph-based representation of version history

The goal of the system is not only to provide functionality but also to serve as a learning tool that exposes internal mechanisms often hidden in traditional systems.

1.2 Objectives

The objectives of the project are as follows:

- To develop a structured system for tracking file changes over time
- To implement efficient storage using content-based identification
- To design a staging and commit workflow
- To analyze differences between file versions using algorithmic approaches
- To optimize storage using delta compression techniques
- To visualize internal processes such as hashing, storage, and reconstruction
- To integrate multiple modules into a cohesive system

2. Introduction

2.1 Background and Context

In modern computing environments, files undergo continuous modifications. Whether in software development, research, or data processing, maintaining different versions of files is crucial. Without a structured approach, tracking changes becomes error-prone and inefficient. Systems that manage file versions typically rely on several key concepts:

- Identifying file content uniquely
- Storing changes efficiently
- Tracking relationships between versions
- Allowing users to inspect and revert changes

However, most systems prioritize usability over transparency. As a result, the internal processes : such as how data is stored or reconstructed, remain hidden from users. GitLite is designed to address this gap by providing both functionality and visibility.

2.2 Problem Statement

The main problem addressed by this project is the lack of transparency and educational clarity in file version tracking systems.

Specifically, the system aims to solve:

- Inefficient storage due to redundant file copies
- Difficulty in understanding differences between versions
- Lack of visibility into internal data structures
- Absence of tools to visualize system behavior

2.3 Scope of the Project

The scope of GitLite covers the complete lifecycle of file version management, from initial input to storage, analysis, and visualization. The system is designed as a self-contained environment that allows users to not only manage file versions but also explore the internal mechanisms behind version tracking systems.

1. File Input and Management Scope

The system supports flexible file handling through:

- Uploading files directly from the user's local system
- Reusing previously uploaded files from an internal repository library

This ensures that users can efficiently work with both new and existing data without redundancy. The system also maintains consistency by ensuring that identical file content is recognized and handled appropriately.

2. Workspace and Editing Scope

GitLite provides an integrated workspace where users can:

- View and edit file content directly within the system
- Perform continuous modifications without external tools

This eliminates the need for switching between multiple environments and ensures that all changes are captured accurately before further operations.

3. Version Tracking Scope

The system supports a structured version control workflow consisting of:

- Staging files before committing
- Creating commits that represent snapshots of file states
- Maintaining a chronological and relational history of versions

Each version is uniquely identified and linked to previous versions, allowing users to track the evolution of files over time.

4. Difference Analysis Scope

A major component of the system is its ability to analyze changes between file versions. This includes:

- Line-by-line comparison of file contents
- Identification of inserted, deleted, and modified lines
- Structured representation of differences

To achieve this, the system incorporates multiple algorithms for comparison, allowing evaluation of performance and accuracy:

- Longest Common Subsequence (LCS)
- Mayers Diff
- Patience Diff
- Histogram Diff

This functionality enables users to understand how a file has evolved across versions while also providing flexibility in choosing different algorithmic approaches.

5. Visualization Scope

One of the defining features of GitLite is its strong emphasis on visualization. The system includes:

- Graphical representation of version history through commit graphs
- Visualization of hash generation processes (Multiple hashing approaches are included for comparison and extensibility: SHA-1, SHA-256, Polynomial Rolling Hash)
- Representation of internal storage structures
- Animated views of delta chain reconstruction

These visual tools transform complex backend operations into intuitive, understandable representations, making the system highly educational and user-friendly.

6. Storage and Optimization Scope

The system implements advanced storage strategies to ensure efficiency:

- Content-based storage where identical data is stored only once

- Delta compression to store only differences between versions
- Organized object storage for efficient retrieval

Additionally, the system includes garbage collection mechanisms to remove unused or redundant data, ensuring optimal use of storage resources.

7. Internal Process Transparency Scope

Unlike traditional systems that abstract away internal operations, GitLite explicitly exposes:

- How file content is converted into unique identifiers
- How data is stored and retrieved
- How versions are reconstructed from stored differences

This transparency allows users to gain a deeper understanding of system behaviour and underlying algorithms.

8. Command and Interaction Scope

The system provides a set of operations that allow users to interact with it effectively:

- Checking the current status of files and versions
- Staging and committing changes
- Performing difference analysis
- Cleaning and optimizing storage

These operations are accessible through an interactive interface that connects user actions to backend processing.

9. Limitations of Scope

While the system covers a wide range of functionalities, it is intentionally limited to a local environment. The following features are outside the current scope:

- Remote repository synchronization
- Distributed version control
- Real-time multi-user collaboration
- Conflict resolution in concurrent edits

These limitations define the boundaries of the project and provide opportunities for future enhancements.

10. Educational and Analytical Scope

Beyond functionality, the system is designed as a learning platform. It allows users to:

- Observe how algorithms operate on real data
- Understand storage optimization techniques

- Visualize abstract concepts such as hashing and graph traversal

This makes GitLite not only a tool for version tracking but also a system for exploring and understanding computational concepts.

3. System Requirements

3.1 Functional Requirements

The following functional requirements define the expected behavior of the system in terms of inputs, processing, and outputs.

1. Input Handling Requirements

- The system shall accept file input in text format from the user.
- The system shall validate file input before processing.
- The system shall maintain a reference to previously loaded files for reuse within the same session.

2. State Management Requirements

- The system shall maintain the current working state of each file.
- The system shall track whether a file is modified, staged, or committed.
- The system shall ensure consistency between the working state and stored versions.

3. Version Creation Requirements

- The system shall generate a new version only when explicitly triggered by the user.
- The system shall associate each version with a unique identifier.
- The system shall maintain parent-child relationships between versions.

4. Data Consistency Requirements

- The system shall ensure that identical file content maps to the same identifier.
- The system shall verify stored data using hash validation before retrieval.
- The system shall prevent corruption of stored objects during read/write operations.

5. Difference Computation Requirements

- The system shall compute differences between any two selected versions.
- The system shall support multiple internal algorithms for comparison.
- The system shall produce deterministic output for identical inputs.

6. Storage Behaviour Requirements

- The system shall store data in a structured format that supports efficient lookup.
- The system shall reuse existing data when identical content is detected.
- The system shall support incremental storage using delta representations.

7. Retrieval and Reconstruction Requirements

- The system shall reconstruct a file version from stored data when requested.
- The system shall correctly apply delta chains to rebuild file content.
- The system shall ensure reconstruction accuracy matches the original version.

8. Optimization Requirements

- The system shall periodically identify unused or unreachable data.
- The system shall remove redundant storage elements without affecting valid data.
- The system shall maintain storage efficiency over time.

9. Visualization Data Requirements

- The system shall provide structured data for visualization modules.
- The system shall expose internal states such as commit relationships and storage mapping.
- The system shall ensure synchronization between backend data and visualization output.

10. Execution Control Requirements

- The system shall process operations in a defined sequence (input → stage → commit → store).
- The system shall prevent invalid transitions between states.

11. Output and Feedback Requirements

- The system shall provide clear output for each operation.
- The system shall generate error messages for invalid inputs or operations.
- The system shall confirm successful execution of tasks.

12. Extensibility Requirements

- The system shall allow integration of additional algorithms for hashing and difference computation.
- The system shall support extension of storage and visualization modules without major restructuring.

4. Design and Implementation

4.1 Algorithms Implementation

The GitLite system uses multiple algorithms for difference computation, hashing, storage optimization, and history management. These algorithms are integrated into different modules to ensure efficiency, accuracy, and scalability.

1. Difference Analysis Algorithms

The system implements multiple diff algorithms to compare file versions and evaluate performance trade-offs.

Algorithms Used:

- **Longest Common Subsequence (LCS):** Identifies the longest matching sequence of lines between two versions to produce accurate and minimal differences.
- **Myers Diff Algorithm:** Computes the shortest edit script between two sequences efficiently, widely used for optimal diff generation.

- **Patience Diff Algorithm:** Uses a patience sorting approach to produce more human-readable diffs by prioritizing unique lines.
- **Histogram Diff Algorithm:** Groups and compares line frequencies to improve diff performance on large files with repeated patterns.

2. Hashing Algorithms

Hashing is used to uniquely identify file content and ensure data integrity within the system.

Algorithms Used:

- **SHA-1** – Used for generating primary identifiers for commits and objects.
- **SHA-256** – Provides a more secure alternative for hashing.
- **Polynomial Rolling Hash** – Used for fast string-based hashing operations.

Working:
- File content is converted into a hash value
- This hash is used as a key for storage and retrieval
- Identical content produces identical hashes (deduplication)

3. Delta Compression Strategy

The system uses a hybrid storage approach based on file size and change characteristics.

Implementation Idea:
- If the file change is small → store as delta (difference from previous version)
- If the file is large or exceeds a threshold → store as a full snapshot

Benefits:
- Reduces storage space
- Balances reconstruction speed and memory usage

4. Commit Storage Using Hashes

Commits are stored using a content-addressable storage model:
- Each commit is assigned a unique hash
- Commits are stored in directories based on hash prefixes
- The hash determines the storage location

Ensures:
- Fast lookup
- Organized storage
- Reduced directory overload

5. Commit Graph Algorithm
- Each commit is represented as a node
- Parent-child relationships form edges
- The structure forms a Directed Acyclic Graph (DAG)
- Enables: Branching ,History traversal ,Visualization

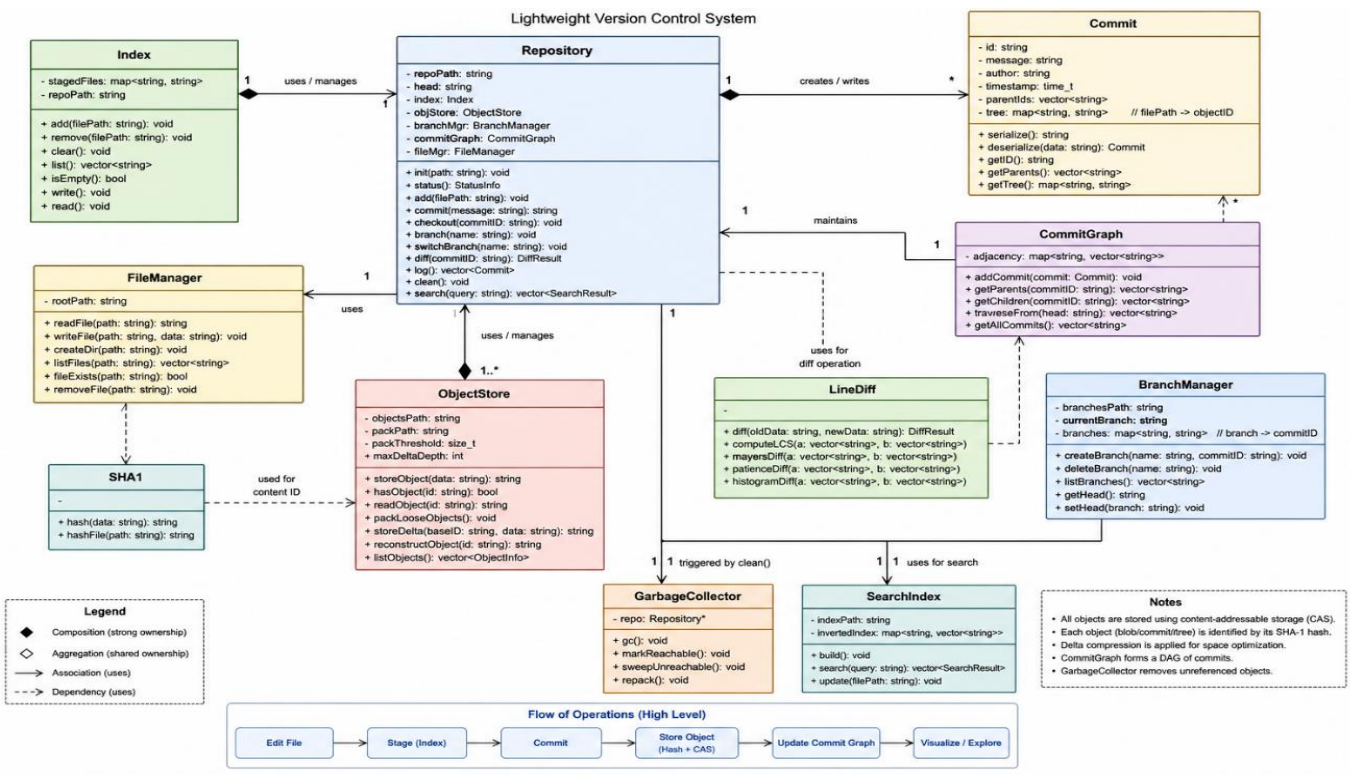
6. Garbage Collection Algorithm

- Traverses stored objects
- Identifies unreferenced commits or objects
- Removes them to optimize storage

4.2 Algorithms Complexities

Algorithm	Time Complexity	Purpose
LCS	O(n × m)	Accurate diff computation
Myers Diff	O((n + m)D)	Optimal edit distance
Patience Diff	O(n log n)	Readable diff output
Histogram Diff	~O(n)	Efficient for repeated patterns
SHA-1 / SHA-256	O(n)	Hash generation
Polynomial Hash	O(n)	Fast string hashing
Delta Reconstruction	O(k)	Apply k patches
Commit Graph Traversal	O(V + E)	Traverse history
Garbage Collection	O(N)	Scan stored objects

4.3 UML Diagram



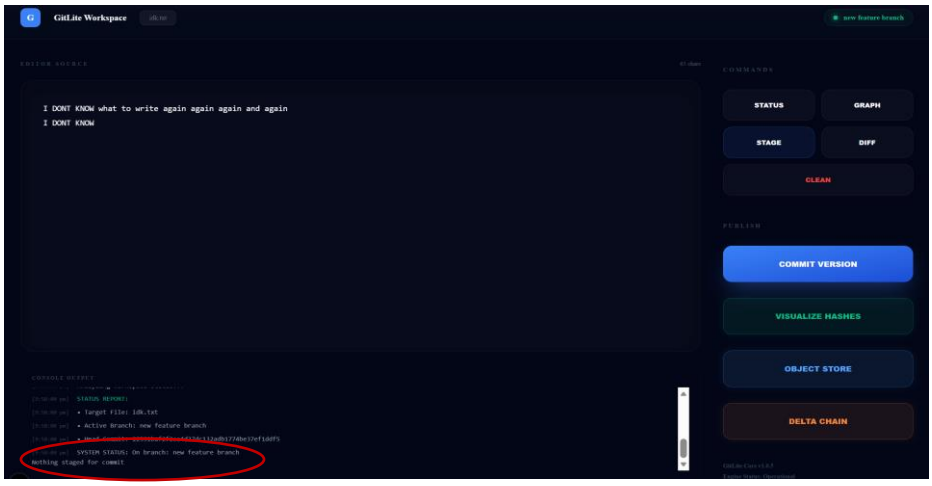
4.4 Output

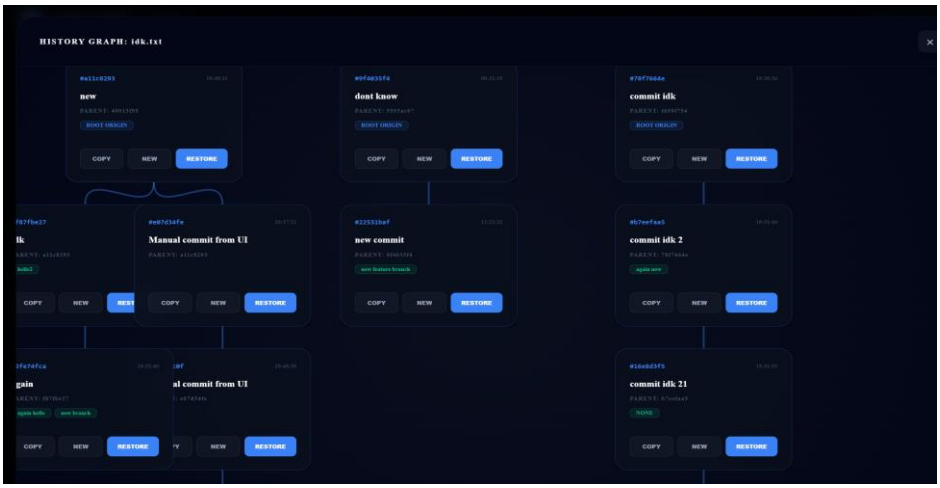
1. GitLite File Input Interface

This screen shows the initial workspace where users can select existing files from the system or import new files from an external source. It acts as the entry point for further operations like editing, staging, and committing.

2. GitLite Workspace and Command Panel

This screen shows the main working environment of GitLite, where files can be edited and managed. The **STATUS** output highlights that the user is currently on the branch **"new feature branch"**, along with details such as the active file, latest commit (HEAD), and staging status.



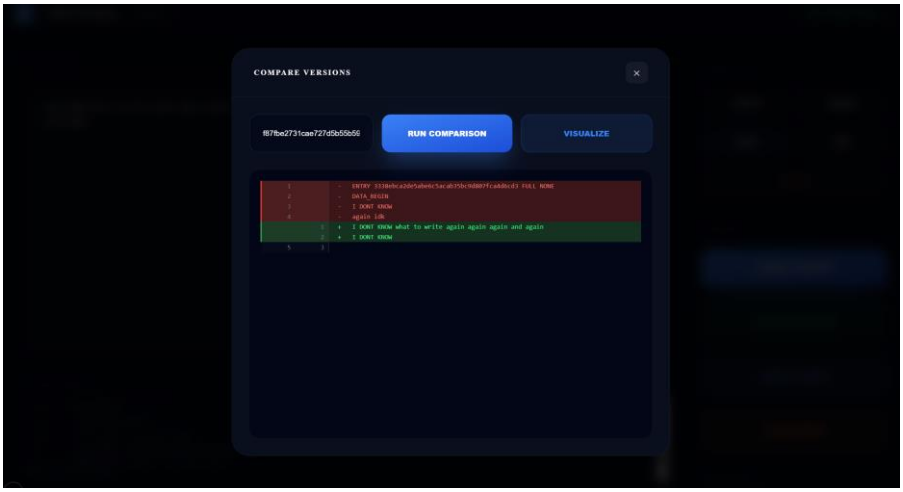


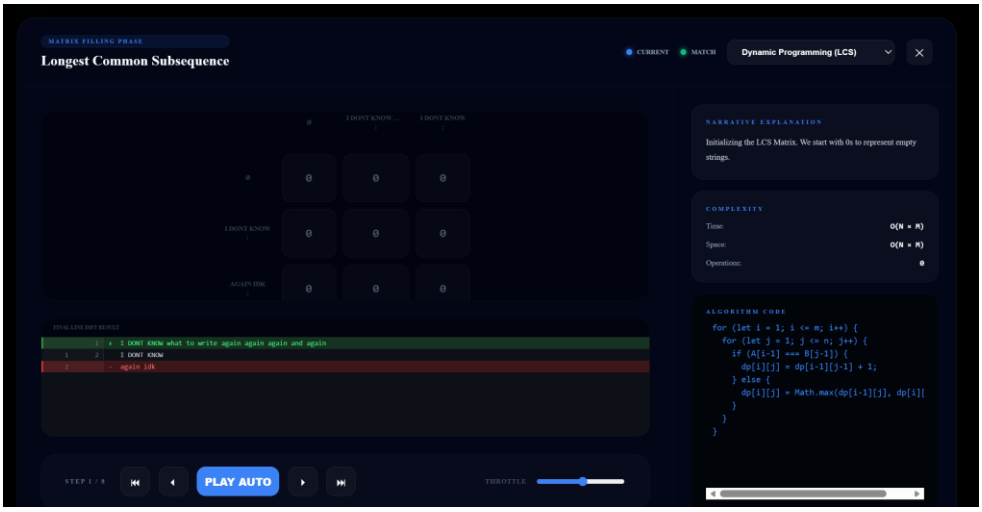
3. Commit History Graph

This screen visualizes the version history as a graph structure, where each node represents a commit and edges show relationships. It allows operations like restore, branch creation, and commit inspection.

4. Difference Comparison Interface

This screen displays the line-by-line comparison between two versions of a file, highlighting additions and deletions for clear understanding of changes.



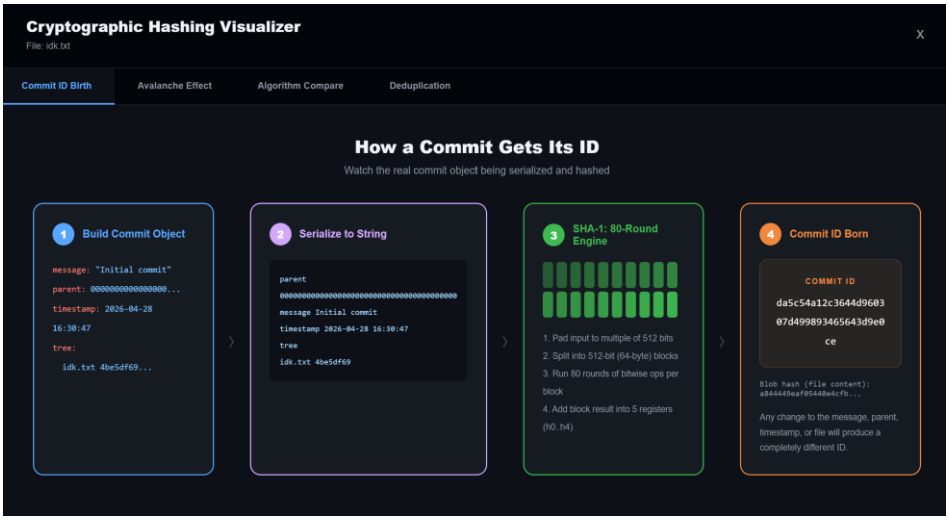


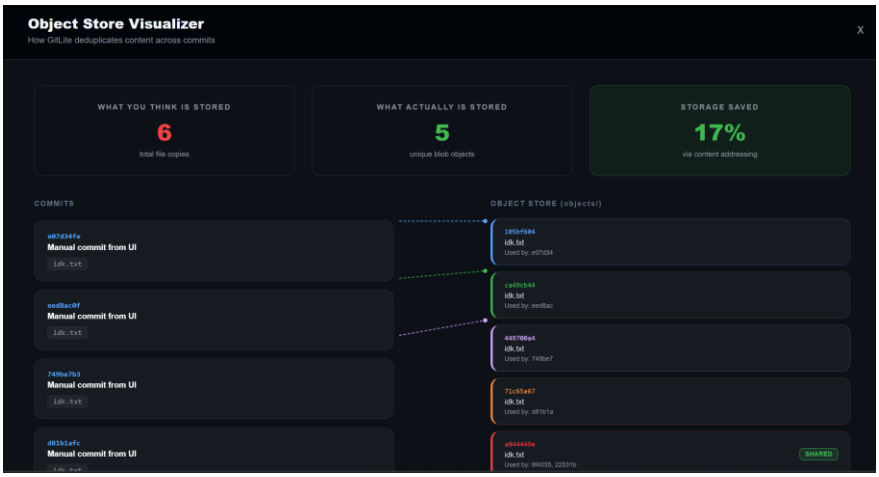
5. LCS Algorithm Visualization

This screen demonstrates the working of the Longest Common Subsequence algorithm, showing matrix computation and final diff output along with complexity details.

6. Hash Visualization

This screen illustrates how a commit ID is generated from file content using hashing, showing serialization, processing steps, and final hash output. The interface also includes additional tabs such as Avalanche Effect, Algorithm Comparison and Deduplication which provide further insights into hashing behavior and storage efficiency.



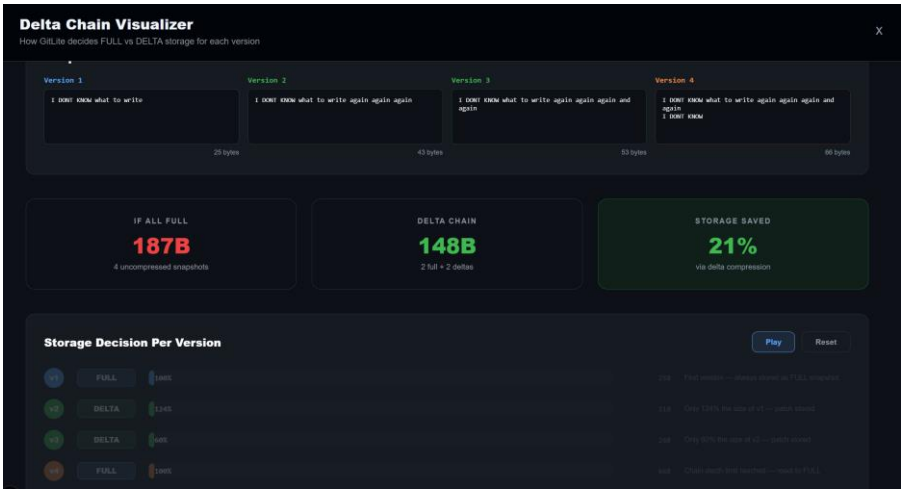


7. Object Store Visualization

This screen shows how file data is stored internally using content-based addressing, highlighting deduplication and storage efficiency.

8. Delta Chain Visualization

This screen demonstrates how versions are stored using a combination of full snapshots and delta differences, along with storage savings achieved.



5. Conclusion

5.1 Summary and Achievements

The project successfully demonstrates:

- File version tracking
- Efficient storage using delta compression
- Difference analysis
- Visualization of internal processes

All objectives have been achieved.

5.2 Future Work

- Add merge functionality
- Improve scalability
- Enhance visualization tools
- Add collaborative features

6. References

- C++ Documentation
- Data Structures and Algorithms textbooks
- Research on hashing and compression
- https://en.wikipedia.org/wiki/Delta_encoding
- https://www.geeksforgeeks.org/sha-1-hash-in-cryptography/